

第10章 泛型与元编程

建议学时:5-7 小时 | **难度:**★★★★★ | **读者背景:**熟悉 C 的 #define 宏、void* 与函数指针,了解 C++ 模板或 Java 泛型(任一即可,不了解也不影响)

🎯 本章学习目标

- 回顾 C 的"泛型"两大困境:预处理宏的文本替换之坑与 void* 的类型丢失
- 掌握泛型函数与泛型结构体,理解 **单态化**:与 C++ 模板相同的"按类型展开"机制,零运行时开销
- 掌握 trait bound 与 where 子句:把"任意类型"变成"满足契约的类型"
- 掌握关联类型(iterator 的 Item)与泛型参数的选型直觉
- 掌握 const 泛型:把"数组长度"变成类型的一部分,对照 C 的数组衰减
- 掌握 macro_rules! 声明式宏:模式匹配 + 代码生成,理解 vec! 等宏的原理
- 了解过程宏三兄弟(派生/属性/函数宏)与 #[derive] 的编译期原理
- 能在"泛型"与"宏"之间正确选型:类型级复用走泛型,语法级生成走宏

💡 从 C 到 Rust:本章主题如何迁移

C 程序员对"代码复用"有两件武器,都有代价。**宏(#define)**:文本替换,不检查类型,宏参数里出现表达式就踩"双重求值""优先级翻转"的坑,调试时看到的是展开后的天书。**void* + 函数指针**:把类型擦掉,换来通用,代价是丢类型安全——栈可以装任意东西,但取出时要人肉转回正确类型,转错就是 UB。C++ 用模板解决了类型问题,代价是复杂的模板语法与编译错误的天书。

Rust 的泛型把 C++ 模板的"零开销 + 编译期展开"保留下来,把语法与错误信息做成现代形态: `fn f<T: Trait>(x: T)` 读起来像函数,约束写在类型参数上,编译器错误指向具体位置。而宏依然存在(`macro_rules!` 与过程宏),但定位完全不同:宏不再做"假装泛型"的事(那是泛型的活),而是做 **语法层面的代码生成**——`println!` 的格式化、`vec!` 的初始化、`#[derive(Debug)]` 的样板生成。一句话记住本章的分工:**泛型处理"类型参数化",宏处理"语法生成"**。

📦 前置知识自查

- C 的 #define 宏(带参宏)、void*、函数指针、数组参数衰减(数组退化为指针)
- 第6章的 trait 与 trait bound(本章的 <T: Trait> 直接复用)
- 第7章的 serde derive 用法(已接触"宏生成的 impl")
- 第9章的 Result(实验题需要)
- (可选)C++ 模板的"编译期展开"概念,用于对照单态化

🚀 本章学习路线

1. **第一阶段(约1.5小时)**:§10.1–§10.3,从 C 的困境到泛型函数/结构体/trait bound——先会"写约束"
2. **第二阶段(约1.5小时)**:§10.4–§10.5,关联类型与 const 泛型——两个容易混淆的进阶特性
3. **第三阶段(约2小时)**:§10.6–§10.7,声明式宏与过程宏——元编程的世界,理解 derive 原理
4. **验收标准**:能徒手写泛型 Stack + 一个自用的 macro_rules!,并能解释"为什么这里用泛型、那里用宏"

本章知识导图

- §10.1 C 的泛型困境:宏与 void* 的代价
- §10.2 泛型函数与泛型结构体:单态化
- §10.3 trait bound 与 where:约束的力量
- §10.4 关联类型:Iterator 的 Item
- §10.5 const 泛型:长度成为类型
- §10.6 macro_rules! 声明式宏
- §10.7 过程宏与 #[derive] 的原理
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§10.1 C 的泛型困境:宏与 void* 的代价

10.1.1 宏:文本替换的灾难

```
// C:用宏"假装泛型"
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int x = MAX(3, 4);           // 看起来正常
int y = MAX(++i, j);         // 爆炸:i 被求值两次!
double z = MAX(1.5, 2);      // 类型不匹配:静默转换

// 更糟:宏没有类型,定义 MAX 时不检查 a/b 是否为数值
#define SWAP(a, b) do { typeof(a) t = (a); (a) = (b); (b) = t; } while (0)
// typeof 是 GNU 扩展,标准 C 根本写不了通用 SWAP
```

10.1.2 void*:擦掉类型的代价

```
// C:void* 栈——装得下一切,也认不出一切
struct Stack { void **data; int top; int cap; };

void stack_push(struct Stack *s, void *v) { /* ... */ }
void *stack_pop(struct Stack *s) { /* ... */ }

// 使用方必须记住"我装的是什么类型",转错就是 UB
struct Point *p = stack_pop(s);           // 若实际装的是 int*——灾难
// 取出时的类型转换没有任何检查,全靠约定
```

两个方案的共同问题: **编译器无法帮你检查"类型契约"**。宏在编译前替换,void* 在编译后消失。Rust 的泛型在"编译期、类型检查之后"展开——既保留类型信息,又生成专用代码。

C 方案	问题	Rust 对应
#define 宏模拟泛型	文本替换、双重求值、无类型检查	泛型 + 单态化(编译期展开,类型全程在场)
void* 通用容器	取出时类型丢失,转错 UB	Vec<T> / 泛型结构体(类型在签名里)
函数指针 + 回调	签名不携带类型约束	trait bound + 闭包(第4/6章)
宏生成重复代码	无法调试、无作用域	macro_rules! / 过程宏(语法级生成,可见可调)

§10.2 泛型函数与泛型结构体:单态化

10.2.1 泛型函数

```
// 一个函数,适用于"任何类型 T"(没有约束时 T 可以做任何类型)
fn first<T>(list: &[amp;T]) -> &T {
    &list[0]
}

fn main() {
    let nums = vec![10, 20, 30];
    let words = vec!["a", "b"];
    println!("{}", first(&nums));    // T = i32
    println!("{}", first(&words));   // T = &str — 同一函数两种化身
}
```

10.2.2 泛型结构体:类型安全的通用容器

```
// 泛型栈:内部是 Vec,对外是栈语义(对照 C 的 void* 栈)
struct Stack<T> {
    items: Vec<T>,
}

impl<T> Stack<T> {
    fn new() -> Stack<T> {
        Stack { items: Vec::new() }
    }

    fn push(&mut self, v: T) {
        self.items.push(v);
    }

    fn pop(&mut self) -> Option<T> {
        self.items.pop()           // 空栈返回 None,不用哨兵
    }

    fn is_empty(&self) -> bool {
        self.items.is_empty()
    }
}

fn main() {
    let mut ints: Stack<i32> = Stack::new();
    ints.push(1);
    ints.push(2);
    println!("弹出:{}", ints.pop().unwrap());

    let mut names: Stack<String> = Stack::new();
    names.push(String::from("张三"));
    // names.push(42);    // ERROR:i32 塞不进 Stack<String> — 类型安全!
}
```

10.2.3 单态化:编译期展开,零开销

单态化(monomorphization):编译器为每个实际用到的 T 生成一份专用代码。`Stack<i32>` 与 `Stack<String>` 编译成两份完全独立的实现,调用处直接内联跳转——与手写两份专用结构体性能完全相同,零虚表、零间接调用。这正是 C++ 模板的机制;Rust 的改进是约束清晰、错误可读。代价:代码体积随"类型 × 函数"组合增长(与 C++ 相同),极端场景用 `&dyn Trait`(第6章)换取单份代码。

§10.3 trait bound 与 where:约束的力量

10.3.1 无约束泛型的边界

```
// 错误:不是所有 T 都能比较(第2章:无隐式强转、无默认比较)
// fn largest<T>(list: &[T]) -> T { let mut m = list[0]; ... m > other ... }
// ERROR: binary operation `>` cannot be applied to type `T`

// 正确:要求 T 可比较(PartialOrd)且可复制(Copy,避免移动语义纠缠)
fn largest<T: PartialOrd + Copy>(list: &[T]) -> T {
    let mut max = list[0];
    for &item in list.iter() {
        if item > max {
            max = item;
        }
    }
    max
}

fn main() {
    let nums = vec![3, 9, 5, 7];
    println!("最大:{}", largest(&nums));           // 9
    let chars = vec!['a', 'z', 'k'];
    println!("最大:{}", largest(&chars));           // z
    // largest(&vec![true, false]);                // bool 没实现 PartialOrd? 有,但顺序出乎意料——约束已经把"能做
    什么"说清楚了
}
```

10.3.2 where 子句:复杂约束的可读写法

```
// 多个约束挤在尖括号里难读;生产惯例:签名简洁,约束挪到 where
fn summarize<T, U>(a: &T, b: &U) -> String
where
    T: std::fmt::Display,
    U: std::fmt::Display + std::fmt::Debug,
{
    format!("{}", vs {:?})", a, b)
}

fn main() {
    println!("{}", summarize(&42, &"x"));          // 42 vs "x"
}
```

⚠ 易错点 1:约束过多,泛型退化成"脆弱的接口"

约束是契约,也是负担:每加一个 bound,调用方能传的类型就少一类。生产经验:先写最少约束,报错时再加——编译器会告诉你"T 需要实现哪个 trait",照着补即可。反之,为了一处小需求给整个函数挂 5 个 bound,会让函数失去复用价值。

§10.4 关联类型:Iterator 的 Item

10.4.1 为什么 Iterator 不用泛型参数

```
// 标准库 Iterator 的简化定义:Item 是"关联类型"
pub trait Iterator {
    type Item; // 关联类型:一个 trait 一个"产出类型"
    fn next(&mut self) -> Option<Self::Item>;
}

// 实现者指定 Item;调用方直接使用
struct Counter { n: u32 }
impl Iterator for Counter {
    type Item = u32; // 承诺:产出 u32
    fn next(&mut self) -> Option<Self::Item> {
        self.n += 1;
        if self.n < 5 { Some(self.n) } else { None }
    }
}

fn main() {
    let mut c = Counter { n: 0 };
    for x in c.by_ref().take(3) { // by_ref:避免消耗计数器
        println!("{}", x);
    }
    // 所有迭代器适配器(sum/filter/map)无需重复写明 Item
    let total: u32 = Counter { n: 0 }.sum();
    println!("总和:{total}"); // 1+2+3+4 = 10
}
```

★ 重点:泛型参数 vs 关联类型,怎么选

同一个 trait 的多个实现,"产出类型"应该用关联类型(Iterator 的 Item);"输入类型"用泛型参数(如第6章的 `Add<Rhs>`——加法可以对接不同类型的右操作数)。直觉:一种类型只产一种 Item,但可以加多种类型的数。一个实现对应"唯一产出"→关联类型;一个实现对应"多组输入组合"→泛型参数。

§10.5 const 泛型:长度成为类型

10.5.1 C 的数组衰减

```
// C:数组当参数就退化成指针,长度信息丢失
int sum_arr(int arr[], int n); // n 必须靠人传,传错没人管
// int sum_arr(int arr[10]); // 写 10 也没用:编译器忽略!
// 想要"长度也类型安全"?C 没有答案(除非宏或模板)
```

10.5.2 const 泛型:长度是类型的一部分

```
// N 是"值参数",在类型层面约束:[u32; N] 长度与 N 绑定
fn sum_arr<const N: usize>(arr: [u32; N]) -> u32 {
    let mut total = 0;
    for &x in arr.iter() {
        total += x;
    }
    total
}

fn main() {
    let a = [1, 2, 3];
    let b = [4, 5, 6, 7, 8];
    println!("{}", sum_arr(a), sum_arr(b));    // N 自动推导 3 和 5

    // 长度错配是编译错误,不是运行期约定
    // let c: [u32; 4] = [1, 2, 3];    // ERROR:长度不符
}
```

对照:泛型参数是"类型",const 泛型是"值(且必须是编译期常量)". 典型用途:固定大小缓冲区、硬件寄存器数组、编译期查表——C 里"宏定义长度"的不变量,在 Rust 里成为类型系统的一部分,长度错配直接编译失败。

§10.6 macro_rules! 声明式宏

10.6.1 模式匹配驱动的代码生成

```
// vec![1, 2, 3] 的真身:标准库宏。
// 声明式宏 = 模式匹配 + 替换,工作方式类似"按语法树匹配的模板"

// 自己实现一个 mini_vec!:匹配"零个或多个表达式,逗号分隔"
macro_rules! mini_vec {
    // 模式:$( $x:expr ),* — 任意多个 expr,用逗号分隔
    ($( $x:expr ),*) => {{
        let mut v = Vec::new();
        $( v.push($x); )*           // 对每个匹配项生成一条 push
        v
    }};
    // 第二个分支:尾部逗号的情况 [1, 2,]
    ($( $x:expr ),* ,) => {
        mini_vec! [ $( $x ),* ]    // 宏内可递归调用
    };
}

fn main() {
    let v = mini_vec![10, 20, 30];    // 展开成 push 三条
    println!("{:?}", v);
    let w = mini_vec![1, 2,];        // 尾逗号也接受
    println!("{:?}", w);
}
```

10.6.2 用宏消灭重复样板

```
// 生产场景:为多个错误变体生成一致的 Display 实现
macro_rules! impl_error {
    ($name:ident, $msg:literal) => {
        impl std::fmt::Display for $name {
            fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
                write!(f, $msg)
            }
        }
        impl std::error::Error for $name {}
    };
}

#[derive(Debug)]
struct TimeoutError;
#[derive(Debug)]
struct BadInputError;

impl_error!(TimeoutError, "操作超时");
impl_error!(BadInputError, "输入非法");

fn main() {
    println!("{}", TimeoutError);    // 操作超时
    println!("{}", BadInputError);  // 输入非法
}
```

⚠ 易错点 2:宏不是函数——优先级、卫生性与调试

宏的展开是"语法树替换"而非"函数调用":参数求值时机由你的模式决定(避免 C 宏的双重求值:每个 `$x` 只展开一次,天然卫生);宏引入的标识符在展开处可见(卫生性限制:局部变量用 `$$` 或小心命名)。调试用 `cargo expand` 看展开结果。规则:能用函数/泛型解决的,别用宏;宏只用于"语法必须变"的场合。

§10.7 过程宏与 `#[derive]` 的原理

10.7.1 过程宏三兄弟

种类	形态	例子
派生宏(derive)	<code>#[derive(Trait)]</code> 生成 impl	<code>#[derive(Debug, Clone, PartialEq)]</code>
属性宏	<code>#[attr]</code> 改写所在条目	<code>#[test]</code> 生成测试入口、 <code>#[tokio::main]</code>
函数宏	名字像函数,生成代码	<code>println!</code> 、 <code>serde_json::json!</code>

10.7.2 #[derive(Debug, Clone)] 到底做了什么

```
// 你写的:
#[derive(Debug, Clone, Copy)]
struct Point { x: f64, y: f64 }

// 编译器看到的(示意):derive 宏读结构体定义,生成样板 impl
// impl std::fmt::Debug for Point {
//     fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
//         f.debug_struct("Point")
//             .field("x", &self.x)
//             .field("y", &self.y)
//             .finish()
//     }
// }
// impl Clone for Point { fn clone(&self) -> Self { *self } }
// impl Copy for Point {}

fn main() {
    let p = Point { x: 1.0, y: 2.0 };
    println!("{:?}", p);           // Point { x: 1.0, y: 2.0 }
    let q = p;                     // Copy:拷贝而非移动
    println!("{:?}", q);
}
```

derive 宏在编译期把结构体的字段列表喂给宏代码,宏输出对应的 impl——字段变更时,生成代码自动跟随,不存在"手写 impl 忘了加字段"的漂移。serde 的 #[derive(Serialize, Deserialize)](第7章)就是同一机制:宏读取字段与属性(rename_all 等),生成完整的序列化代码。自己写过程宏需要独立的 proc-macro crate,入门后用 serde/thiserror 这类成熟宏即可——但理解"宏在编译期读你的代码、生成你的代码",是读懂它们文档的前提。

本章总纲:泛型 vs 宏

泛型:参数是"类型",产出"类型的组合",检查在类型层面。宏:参数是"代码(token)",产出"代码",检查在语法层面。判据:你的抽象能不能用"一个类型参数"表达?能,就泛型;需要"语法变形"(生成 impl、改写函数、编译期 DSL),就宏。两者都做"编译期的事"——这是与 C 的运行时黑客最根本的分野。

易错点 3:泛型与 dyn 混用的"静态/动态"错觉

fn f<T: Trait>(x: T) 与 fn g(x: Box<dyn Trait>) 都能接"实现了 Trait 的类型",但前者是单态化(每个 T 一份代码,可内联),后者是 vtable 查找(一份代码,不可内联)。同一个接口里来回切换两者,是设计混乱的开始:先问"调用点的具体类型是否编译期确定",确定走泛型,不确定走 dyn——选定后保持全链路一致。

本章速查表

主题	结论 / 写法
泛型函数	<code>fn first<T>(list: &[T]) -> &T</code> —— 调用时推断 T
泛型结构体	<code>struct Stack<T> { items: Vec<T> };impl<T> Stack<T> { ... }</code>
单态化	编译期为每个具体类型生成专用代码,零运行时开销(同 C++ 模板)
trait bound	<code>fn f<T: PartialOrd + Copy>(…)</code> —— 多约束用 +
where 子句	约束挪到函数签名下方,复杂约束可读性优先
关联类型	<code>trait Iterator { type Item; ... }</code> —— "产出类型"用关联类型
泛型参数 vs 关联	输入类型用泛型参数(<code>Add<Rhs></code>);唯一产出用关联类型(<code>Item</code>)
const 泛型	<code>fn f<const N: usize>(a: [u32; N])</code> —— 长度成为类型的一部分
声明式宏	<code>macro_rules! name { (\$(\$x:expr),*) => {...} }</code> 模式匹配 + 生成
宏重复	<code>\$(code)*</code> 对匹配的每一项生成一段代码
过程宏	<code>derive</code> (生成 <code>impl</code>)/ 属性(改写条目)/ 函数(生成代码)三类
derive 原理	编译期读结构体字段,生成 <code>impl</code> ;字段变更自动跟随
选型	类型参数化→泛型;语法生成→宏;能函数就别宏

最小必会清单

- 能解释单态化的机制与开销,并说出与 C++ 模板的一致性
- 能徒手写泛型结构体 `Stack<T>` 及 `impl` 块
- 能写 `trait bound`(含多约束)与 `where` 子句,并解释各自适用场景
- 能说出关联类型与泛型参数的选型原则(产出 vs 输入)
- 能写出 `const` 泛型函数并解释它如何消灭"数组长度靠约定"
- 能解释宏与函数/泛型的区别(语法树替换 vs 调用/类型展开)
- 能读懂与编写简单的 `macro_rules!`(至少 `mini_vec` 级别)
- 能说出过程宏三兄弟与 `#[derive]` 的编译期原理
- 知道调试宏展开用 `cargo expand`
- 能在"泛型 vs 宏"之间做出正确选型

自测题

Q 1. 单态化与虚函数调用(第6章 `dyn`)的开销差异是什么?分别适合什么场景?

✅ 参考答案

单态化:编译期生成专用代码,调用直接内联,零间接跳转;代价是代码体积随"类型×函数"组合增长。dyn:运行期查 vtable,一次间接调用,代码单份;代价是无法内联与特化。选型:类型集合编译期已知、性能敏感(数值库、容器)→泛型;类型集合运行期才闭合(插件、异构集合)→dyn。

Q 2. C 的 #define MAX(a,b) 与 Rust 的泛型函数,本质区别在哪?

✓ 参考答案

宏是预处理期的文本替换:不参与类型检查,参数被多次求值(MAX(++i,j) 的经典爆炸),错误在展开后爆发且难定位。泛型是编译期的类型级抽象:参数有类型,约束由编译器检查,每个展开点都经过完整类型检查,错误指向源代码位置。宏"假装泛型"的时代结束了——泛型负责类型,宏退守语法。

Q 3. trait Iterator 为什么用关联类型 Item 而不是泛型参数?

✓ 参考答案

因为"产出类型"对每个实现是唯一的:Counter 只产出 u32,不可能产出 u32 又产出 String。用关联类型,一个 impl 块声明一次 Item,所有方法(next 等)自动共享,调用方无需重复标注。若用泛型参数,一个类型要产出多类元素才能表达,语义错位且所有适配器都要多写一个参数。

Q 4. 数组 [u32; N] 与 C 的数组参数有什么本质差异?const 泛型解决了什么?

✓ 参考答案

C 的数组参数退化为指针,长度信息丢失,只能靠额外的 n 参数靠约定传递。Rust 的 [u32; N] 中长度是类型的一部分:sum_arr<const N: usize>(arr: [u32; N]) 让 N 成为编译期参数,长度错配直接编译错误——"这个数组一定是 4 个元素"从注释变成类型事实,编译器替你维护不变量。

Q 5. macro_rules! 与函数,什么时候必须用宏?

✓ 参考答案

函数在"值层面"复用:输入是值、输出是值。当复用点是"语法形态"时必须用宏:① 生成 impl/derive 样板;② 接受"不限数量、不限类型的参数"(println! 的格式化串);③ 需要在调用点展开上下文代码(第3章的标签、泛型数组字面量)。判断口诀:数据进函数,代码进宏。

Q 6. #[derive(Debug)] 之后给结构体加了一个字段,会发生什么?手写 impl 呢?

✓ 参考答案

derive 生成的 impl 在编译期按当前字段列表重建:加字段后 Debug 输出自动包含新字段,零手工维护。手写 impl Debug 则必须自己加字段的格式化代码,漏加就是"打印与真实结构不一致"的隐蔽漂移。这就是"生成代码跟随类型定义"的价值——也是 serde/thiserror 等宏生态在生产中无可替代的原因。

章末实验题:泛型栈与自用宏

实验 10:泛型栈与自用宏

实验目标:实现一个泛型 Stack(含 trait bound 方法),再用 macro_rules! 写一个"简化初始化"宏,并加一个 const 泛型函数,把本章全部特性串起来。

实验要求:

- 任务 1:泛型 Stack<T>,含 new/push/pop/peek/is_empty(覆盖:泛型结构体 §10.2)
- 任务 2:为 Stack<T> 实现 Display——要求 T: Display,输出栈内元素(覆盖:trait bound §10.3)
- 任务 3:实现 PartialEq——要求 T: PartialEq,两个栈逐元素比较(覆盖:多约束/where §10.3)
- 任务 4:写 stack! 宏:stack![1, 2, 3] 生成带元素的栈(覆盖:macro_rules! §10.6)
- 任务 5:const 泛型函数 fixed_sum<const N: usize> 求 [i32; N] 之和(覆盖:const 泛型 §10.5)
- 加分:用 #[derive(Debug, Clone, PartialEq)] 定义测试用结构体,验证 derive 生成(覆盖:derive 原理 §10.7)

第一步 一步:写 Stack<T> 的骨架与四个基础方法,先跑通 push/pop

第二步 二步:实现 Display:先加 T: Display 约束编译,再把约束换成 where 形式感受差异

第三步 三步:实现 PartialEq(需要 T: PartialEq),并体会"实现 std trait 就是运算符可用"

第四步 四步:写 stack! 宏(注意:生成的是带初始值的 Stack,需要先有 new/push)

第五步 五步:写 const 泛型函数,并尝试用不同长度数组调用验证类型绑定

参考答案要点

```
// ===== 实验 10:泛型栈与自用宏 =====
// 覆盖知识点:泛型结构体与 impl($10.2)、trait bound 与 where($10.3)、
//          const 泛型($10.5)、macro_rules!($10.6)、derive($10.7)

use std::fmt;

// ---- 任务 1:泛型栈 ----
#[derive(Debug, Clone)]
struct Stack<T> {
    items: Vec<T>,
}

impl<T> Stack<T> {
    fn new() -> Stack<T> {
        Stack { items: Vec::new() }
    }

    fn push(&mut self, v: T) {
        self.items.push(v);
    }

    fn pop(&mut self) -> Option<T> {
        self.items.pop()
    }

    fn peek(&self) -> Option<&T> {
        self.items.last()
    }

    fn is_empty(&self) -> bool {
        self.items.is_empty()
    }
}

// ---- 任务 2:Display,需要 T: Display ----
impl<T: fmt::Display> fmt::Display for Stack<T> {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[");
        for (i, item) in self.items.iter().enumerate() {
            if i > 0 {
                write!(f, ", ");
            }
            write!(f, "{item}")?;
        }
        write!(f, "]")
    }
}

// ---- 任务 3:PartialEq,多约束用 where 写清楚 ----
impl<T> PartialEq for Stack<T>
where
    T: PartialEq,
{
    fn eq(&self, other: &Self) -> bool {
        self.items == other.items
    }
}
```

```

    }
}

// ---- 任务 4:stack! 宏:自动调用 new/push ----
macro_rules! stack {
    ($( $x:expr ),* $(,)? ) => {{
        let mut s = Stack::new();
        $( s.push($x); )*
        s
    }};
}

// ---- 任务 5:const 泛型求和 ----
fn fixed_sum<const N: usize>(arr: [i32; N]) -> i32 {
    let mut total = 0;
    for &x in arr.iter() {
        total += x;
    }
    total
}

// ---- 加分:derive 生成的类型,直接可用于断言 ----
#[derive(Debug, Clone, PartialEq)]
struct Point { x: i32, y: i32 }

fn main() {
    // 泛型栈两种类型互不干扰
    let mut s1 = stack![1, 2, 3];
    println!("s1 = {s1}"); // [1, 2, 3]
    assert_eq!(s1.pop(), Some(3));

    let mut s2 = Stack::<String>::new();
    s2.push(String::from("rust"));
    println!("s2 = {s2}"); // [rust]

    // 栈相等比较:需要 T: PartialEq(编译期保证)
    let a = stack![1, 2];
    let b = stack![1, 2];
    assert!(a == b, "同元素栈应相等");

    // const 泛型:长度进入类型
    println!("sum([1,2,3]) = {}", fixed_sum([1, 2, 3]));
    println!("sum([4,5,6,7]) = {}", fixed_sum([4, 5, 6, 7]));

    // derive 产物:Debug/Clone/PartialEq 全部可用
    let p = Point { x: 1, y: 2 };
    let q = p.clone();
    assert_eq!(p, q);
    println!("Point: {p:?}");
    println!("全部断言通过");
}

```

设计说明:任务 2 与任务 3 是"约束生长"的现场——先写 impl 不加约束,编译器报 "T cannot be formatted/compared",补上 bound 才通过。这种"编译器告诉你缺什么"的节奏,正是生产里写泛型的常态。

`stack!` 宏的 `$(,)?` 分支吸收尾逗号,是与标准库 `vec!` 对齐的细节;注意宏内 `Stack::new()` 的路径是调用处可见的——宏卫生性允许它引用模块路径,但要求调用方能看到该类型,这也是宏与函数的一个微妙差异。

下一章预告:第11章 反射与动态特性——Rust 如实说"没有反射";Any 与 `downcast`、`serde` 的编译期序列化、宏生成与 `trait` 动态分派,如何替代运行期自省。